SMART RESTAURANT MANAGEMENT AND CUSTOMER QUEUE OPTIMIZATION SYSTEM

Complete End-to-End Project Blueprint
For Human Understanding and AI-Agent Implementation

Technology: Google Apps Script + Google Sheets + HTML + CSS + JavaScript

System type: Dine-in restaurant management (not food delivery)

Interfaces: Main Website • Customer Website • Staff Website • Kitchen Website • Admin Website

Version 3 — Expanded implementation specification

Written in simple, direct English. Finalized rules are stated as requirements; unresolved decisions are explicitly marked.

How to use this document

This document is the single project brief for rebuilding the system. A developer or AI coding agent should read it from beginning to end before creating files. It explains what the system must do, who uses it, how each website works, how requests travel through Code.gs, how all websites share one Google Spreadsheet, and what must be tested before delivery.

Important distinction: this is a requirements and architecture blueprint. It does not claim that the deleted source code, live Google project, spreadsheet, credentials, or deployment currently exist. Build the system from this specification and verify each stage.

Implementation rules

- Do not change finalized business rules or database columns without asking the project owner.
- Do not invent missing decisions. Put unresolved items in a decision log and ask before implementing them.
- Keep the five websites separate at the interface level, but connect all of them to one Apps Script backend and one shared spreadsheet.
- Keep code modular. Code.gs is the central request entry point, not a file that should contain every business rule.
- Never expose secrets, password hashes, OTP hashes, or private customer data to the browser.
- Build and test locally with safe mocks first. Connect the real Google Spreadsheet only in the approved integration stage.
- Do not add GitHub setup to the current project scope.

Contents

- 1. Project overview and goals
- 2. Fixed technology and architecture
- 3. Five websites and user roles
- 4. Full folder and Apps Script file responsibilities
- 5. Google Sheets creation and database connection
- 6. Exact database schema: 12 tabs and 108 columns
- 7. Authentication, security and access rules
- 8. Complete customer journey: online booking
- 9. Walk-in QR and queue journey
- 10. Table allocation and reservation rules
- 11. Food ordering and kitchen workflow
- 12. Extra items, item changes and cancellation
- 13. Billing, payment, refund and departure
- 14. Feedback, history and reports
- 15. UI and design requirements
- 16. Backend request lifecycle and data consistency
- 17. Error handling and edge cases
- 18. Testing and demo data
- 19. Build order and acceptance checklist
- 20. Decisions that must not be guessed

1. Project overview and goals

The Smart Restaurant Management and Customer Queue Optimization System is a digital system for a dine-in restaurant. It helps customers book or join a queue, helps staff manage tables and orders, helps kitchen staff prepare authorized food, and helps administrators control the restaurant.

Main goals

- Reduce confusion during busy hours by showing bookings, arrivals, queues, table availability and order status in one system.
- Prevent two bookings or staff members from allocating the same table at the same time.
- Give customers a clear way to register, reserve, confirm arrival, order food and view bills.
- Ensure kitchen staff receive only orders that staff have authorized.
- Keep billing and payment verification under staff/admin control.
- Maintain a reliable record of status changes, orders, bills, payments and refunds.

What the system is not

- It is not a food-delivery platform. No delivery partner, delivery address, delivery tracking or delivery fee workflow is required.
- It is not a payment gateway integration. Initial payment handling is manual UPI payment with staff verification.
- It does not use Firebase, Firestore, MySQL, PostgreSQL, MongoDB, Supabase or another database.
- It does not use a separate Node.js/Express backend. Google Apps Script is the backend.

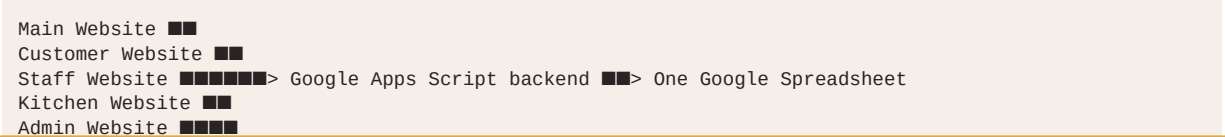
Core system principle

Every customer visit has one Booking_ID, including a walk-in visit with no reservation. A visit can contain multiple Order_IDs, but the visit has one final Bill_ID. Bookings.Visit_Type is the source of truth for Online or Walk-in.

2. Fixed technology and architecture

Layer	Required technology / rule
Frontend	HTML, CSS and vanilla JavaScript. Five separate website interfaces.
Backend	Google Apps Script (V8 runtime).
Database	One Google Spreadsheet containing exactly 12 tabs and 108 defined columns.
Communication	Frontend calls Apps Script server functions through the approved web-app/client integration pattern.
Authentication	Customer email OTP; staff/admin/kitchen password login with server-side sessions.
Payments	Manual UPI reference capture and staff verification. No payment gateway in initial scope.
Development	Local development and mock tests first; live Google connection only during approved integration.

Architecture at a glance



All five interfaces use the same backend and the same spreadsheet. They do not each create their own database. The backend decides what a user is allowed to do; hiding a button in the browser is not security.

Single source of truth

The spreadsheet is the shared persistent data store. Apps Script modules read and write it. Do not create local browser storage or a second database as the official record. Temporary UI state such as an open cart can exist in the browser, but confirmed business records must be saved through the backend.

3. Five websites and user roles

3.1 Main Website — public

- Show restaurant name, introduction, location/contact details, opening information, menu highlights and general information.
- Show a clear Book a Table action that opens the Customer Website.
- Allow visitors to browse public information without signing in.
- Do not expose staff/admin pages or private customer data.

3.2 Customer Website

- Identify a returning customer by mobile number and use email OTP to authenticate the customer.
- Create or update the customer's basic profile where permitted.
- Show menu categories, food details, prices and availability; support search.
- Allow cart management and order submission.
- Show table availability and allow optional table reservation.
- Show the I'm Arrived action, arrival status and queue/call status.
- Show active order progress, bills, payment information and visit history.
- Allow one feedback submission for a completed booking.

3.3 Staff Website

- Show upcoming bookings, walk-ins, arrival confirmations, queue and table status.
- Verify customer arrival and allocate suitable tables.
- Seat customers and update physical table status.
- Create/manage orders for a booking, add extra items and authorize food preparation.
- Handle item adjustments/cancellations under the approved rules.
- Generate the final bill, record payment evidence and verify actual receipt.
- Request refunds and manage customer departure/table release.

3.4 Kitchen Website

- Show only food preparation requests authorized by staff.
- Show order and item details needed for preparation.
- Update preparation/order status and record status history.
- Do not allocate tables, verify customer arrival, generate bills, collect payment or approve refunds.

3.5 Admin Website

- Manage customer records and staff/kitchen accounts.
- Manage menu items, categories, prices, availability and active status.

- Manage tables and seating capacities.
- View and manage bookings, reservations, queues, orders, bills, payments and refunds.
- Manage restaurant settings and view reports.
- Review and approve/reject refund requests.

Role capability boundary

Action	Customer	Staff	Kitchen	Admin
Browse public menu	Yes	Yes	As needed	Yes
Book / join queue	Yes	Can assist	No	Can inspect
Verify arrival / allocate table	No	Yes	No	Admin oversight
Create / authorize prep request	Submit order	Yes	No	Oversight
Update kitchen preparation status	View	View	Yes	View
Generate final bill / verify payment	View/pay instruction	Yes	No	Yes/oversight
Approve or reject refund	Request/view	Request	No	Yes
Manage users/settings/menu/tables	No	No	No	Yes

4. Full folder and Apps Script file responsibilities

Keep the project modular. Exact filenames may be adjusted only if the same responsibilities remain clearly separated. Code.gs is the main entry point and coordinator. It must not become a giant file containing every feature.

4.1 Code.gs — central backend entry point

Code.gs receives requests from the five websites, routes each request to the correct backend function, coordinates authentication and role checks, validates the request envelope, catches unexpected errors and returns a consistent response. It is the front door of the backend.

- Provide the web-app entry point (for example, doGet) to serve the approved application shell or route.
- Expose a controlled server-side request dispatcher for frontend actions.
- Map allowed action names to backend modules; never execute arbitrary client-supplied function names.
- Call authentication/session validation before protected actions.
- Call role/ownership authorization before reading or changing protected records.
- Call feature services for booking, queue, tables, orders, kitchen, billing, payments and refunds.
- Return a consistent success/error response without leaking stack traces, secrets or internal sheet data.
- Log safe audit information where required; never log OTP plaintext, passwords, secrets or full sensitive payloads.

Code.gs should coordinate the modules, not duplicate their business logic. For example, a customer booking request should be routed to BookingService, which uses reservation/table modules and database helpers.

4.2 Suggested Apps Script module map

File / module	Responsibility
Config.js	Only runtime configuration values and approved configuration access.
Config.template.js	Safe example/template showing required settings; contains no real secrets.
Code.gs	Web app entry point, request dispatcher, routing, common response/error boundary.
Database.js	Shared spreadsheet connection helper, safe read/write helpers and database availability checks.
DatabaseSchema.js	Exact 12-tab / 108-column schema definitions and header verification.
DatabaseSetup.js	Explicit setup/initialization functions; no destructive reset by default.
AuthService.js	Customer OTP flow, staff/admin/kitchen login, session creation/validation/logout.
PasswordService.js	Password hashing/verification using approved PBKDF2-HMAC-SHA256 design.
Authorization.js	Role checks, customer ownership checks and protected-action rules.
CustomerService.js	Customer lookup, profile and customer-facing history.
BookingService.js	Create booking IDs and manage booking lifecycle.
ReservationService.js	Reservation records, grace deadlines, no-show/expiry and reservation release.
QueueService.js	Queue ordering, capacity-fit selection and call protection.
TableService.js	Table availability, allocation, seating, occupancy and release.
MenuService.js	Menu listing, filtering and admin menu management.
OrderService.js	Orders, items, staff authorization and item lifecycle.
KitchenService.js	Kitchen queue and preparation status changes.
BillingService.js	Subtotal, tax/discount inputs, adjustments and final bill.
PaymentService.js	Payment records, UPI reference, receipt verification and payment status.
RefundService.js	Refund request, admin review and manual completion tracking.
HistoryService.js	Append status/audit history and retrieve timelines.
ReportService.js	Admin operational and financial reports.
Validation.js	Server-side validation, type/range/required-field checks.
LockService.js or LockHelpers.js	Shared concurrency helpers for critical spreadsheet changes.
Utils.js	ID generation, date/time formatting, safe common helpers.
EmailService.js	OTP email delivery using MailApp and safe email templates.

4.3 Frontend file responsibilities

- Keep each website in its own folder or clearly separated application entry point.
- Use shared CSS/JS components for common formatting only where that does not merge the five role-specific experiences.
- Frontend JavaScript calls approved backend actions and renders returned data.
- Never put passwords, OTP secrets, spreadsheet IDs with privileged access, or role enforcement logic in frontend code.
- Validate forms in the browser for usability, but repeat all validation on the server.

4.4 Example request path

```
Customer clicks “Book Table” → Customer Website JavaScript sends a booking action → Code.gs receives and routes it → AuthService verifies the customer session → Authorization checks customer ownership → BookingService validates booking details → ReservationService / TableService handle reservation logic → Database.js reads/writes the shared spreadsheet → Code.gs returns a safe response → Customer Website updates the screen.
```

5. Google Sheets creation and database connection

5.1 Create the one intended database

1. Use the already-separated project Gmail account chosen by the owner.
2. Create a completely fresh Google Cloud project if a Cloud project is required for the approved deployment. Do not reuse an old wound-project Cloud project or files.
3. Create one new Google Spreadsheet for this restaurant system.
4. Create exactly the 12 tabs and headers specified in Section 6. Do not add extra tabs or columns without approval.
5. Copy the spreadsheet ID from its URL. Treat it as configuration, not a value to hardcode throughout the source.
6. Create the Apps Script project and add the modular .gs files. Configure the approved runtime settings in Config.js.
7. Keep Config.template.js as a safe example only. Do not put real secrets in the template or source repository.
8. Run the read-only schema verification first. Only run the explicit missing-tab/header initializer after the owner confirms the target is the fresh intended spreadsheet.
9. Authorize Google permissions using the owner's account and test read/write access with a harmless test operation.

5.2 Shared connection helper

Database.js must expose one shared getDatabaseConnection() helper. It reads the approved spreadsheet ID setting from Config.js and opens the same spreadsheet using SpreadsheetApp.openById(...). Every service that needs data uses this shared helper instead of opening a different spreadsheet or embedding its own ID.

Conceptual example only — the actual config property name must match the approved Config.js contract:

```
function getDatabaseConnection() {
const spreadsheetId = getApprovedSpreadsheetIdFromConfig();
if (!spreadsheetId) throw new Error('Database is not configured');
return SpreadsheetApp.openById(spreadsheetId);
}
```

Do not copy this conceptual placeholder literally if the actual Config.js contract uses a different property. Keep one authoritative configuration path.

5.3 Important database checks

- isDatabaseConfigured() checks whether the required setting exists and is syntactically usable.
- A separate connection test must actually open the spreadsheet and confirm access; configuration existing does not prove the spreadsheet is reachable.
- verifyDatabase() is read-only: it checks required tabs, exact headers, missing/extra columns and schema consistency.
- initializeMissingDatabaseTabs() must be an explicit, owner-approved action on the intended fresh spreadsheet. It must not clear, overwrite or delete existing data.
- All five websites use the same Apps Script deployment/backend and same spreadsheet configuration.
- Use LockService around critical read-check-write operations such as table allocation, reservation protection, queue calls, bill finalization and refund state transitions.

5.4 Secret configuration

The only approved ScriptProperties secret key is OTP_HMAC_SECRET. Generate a random 32-byte secret locally and enter it manually during the approved Apps Script configuration stage. Never send it in chat, place it in source code, commit it, display it in logs or return it to the browser. Do not add other ScriptProperties keys without explicit owner approval.

5.5 Local testing versus live integration

During early development, use mock database adapters and tests. They must imitate the required data operations without pretending a live spreadsheet is connected. Real SpreadsheetApp.openById access, Google authorization, MailApp delivery and deployment must be tested only in the integration stage with the fresh approved spreadsheet.

6. Exact database schema: 12 tabs and 108 columns

The following is the finalized schema. Keep the tab names and column names exactly as written. The database contains 12 tabs and 108 columns in total. Do not add a separate Feedback tab; feedback fields are stored in Bookings.

Customers

10 columns: Customer_ID, Customer_Name, Mobile_Number, Email, Created_At, OTP_Hash, OTP_Expiry, OTP_Attempts, OTP_Last_Sent_At, OTP_Session_ID

Bookings

14 columns: Booking_ID, Customer_ID, Visit_Type, Booking_Date, Reservation_Date, Reservation_Time, Booking_Status, Party_Size, Queue_Status, Queue_Joined_At, Status_History_JSON, Feedback_Rating, Feedback_Comment, Feedback_Submitted_At

Tables

4 columns: Table_ID, Table_Number, Seating_Capacity, Table_Status

Table_Reservations

7 columns: Reservation_ID, Booking_ID, Table_ID, Reservation_DateTime, Grace_Deadline, Arrival_Status, Reservation_Status

Menu

9 columns: Food_ID, Food_Name, Category, Description, Price, Image_URL, Availability, Prep_Time_Minutes, Is_Active

Orders

6 columns: Order_ID, Booking_ID, Order_DateTime, Order_Status, Authorized_At, Authorized_By

Order_Items

10 columns: Order_Item_ID, Order_ID, Food_ID, Quantity, Unit_Price, Subtotal, Item_Status, Cancellation_Reason, Replacement_Food_ID, Price_Difference

Bills

10 columns: Bill_ID, Booking_ID, Bill_DateTime, Subtotal, Tax_Amount, Discount, Grand_Total, Payment_Status, Discount_Reason, Adjustments_JSON

Payments

22 columns: Payment_ID, Bill_ID, Payment_DateTime, Payment_Method, Amount_Paid, Payment_Status, Transaction_Type, Transaction_Reference, Received_By, Related_Payment_ID, Refund_Status, Requested_Amount, Completed_Refund_Amount, Refund_Reason, Rejection_Reason, Requested_By, Requested_At, Reviewed_By, Reviewed_At, Completed_By, Completed_At, Refund_Reference

Users

8 columns: User_ID, User_Name, Email, Role, Account_Status, Password_Hash, Salt, Active_Sessions_JSON

Restaurant_Settings

2 columns: Setting_Key, Setting_Value

Order_Status_History

6 columns: History_ID, Order_ID, Order_Status, Changed_At, Changed_By, Change_Reason

Schema integrity rules

- IDs must be unique and generated on the server. Do not trust IDs supplied by the browser.

- Use stable date/time formats consistently. Store timestamps in a consistent format and display them in the restaurant's local timezone.
- Use numeric values for quantities, prices and amounts. Validate non-negative values and permitted precision.
- Use exact status values defined by the application; do not allow arbitrary free-text status updates.
- Do not delete financial history to correct mistakes. Use approved adjustment/refund records and audit history.
- One Booking_ID can have multiple Order_IDs; one final Bill_ID is associated with a Booking_ID.

7. Authentication, security and access rules

7.1 Customer authentication

1. Customer enters mobile number to look up an existing profile or start registration.
2. Mobile number is an identifier, not proof of identity. Do not sign the customer in using mobile number alone.
3. Send a six-digit email OTP (100000–999999) to the customer's registered/provided email using MailApp.
4. OTP expires after 5 minutes. Permit a maximum of 3 failed attempts. Resend cooldown is 60 seconds.
5. A newly generated OTP invalidates the previous OTP.
6. Store only an HMAC-SHA256 hash of the OTP using OTP_HMAC_SECRET. Never store or log OTP plaintext.
7. On successful verification, create/refresh a customer session and associate it with the customer account.

7.2 Staff, Admin and Kitchen authentication

- Staff, Admin and Kitchen users sign in with their account email/username and password.
- Store passwords using PBKDF2-HMAC-SHA256 with a unique per-user salt. Never store plaintext passwords.
- Production PBKDF2 iteration count must be benchmarked in the actual Apps Script V8 runtime. Do not assume Node.js benchmark results apply.
- Create server-managed sessions, validate them on every protected request and support logout/session expiry.
- Account_Status must be checked. Disabled accounts must not be able to authenticate or continue protected actions.

7.3 Authorization

- The backend must check the user's role for every protected operation.
- A customer may read or change only their own permitted profile, bookings, orders, bills and history.
- Patient/customer ownership cannot be inferred from a supplied Booking_ID alone; verify the association in the database.
- Staff can perform the operational tasks assigned to Staff in this specification.
- Kitchen can update preparation status only for authorized work; it cannot access payment administration or table allocation controls.
- Admin has management and oversight permissions, including refund approval/rejection.
- Never rely on hiding a button or menu item as access control.

7.4 General protection

- Validate every server input, including IDs, dates, times, roles, quantities, prices, status transitions and ownership.
- Protect critical operations with locks to avoid race conditions.
- Escape/sanitize user-visible text to prevent script injection.
- Return generic user-safe errors; keep technical detail in restricted logs without secrets.

- Record important state changes with actor, time, reason and previous/new state where applicable.

8. Complete customer journey: online booking

This is the full online customer path from opening the website to leaving the restaurant. Some customers may reserve a table; table reservation is optional. A Booking_ID is still created for every visit.

1. Customer opens the Main Website and selects Book a Table.
2. Customer Website opens. Customer enters mobile number.
3. System looks up the customer profile. For a new customer, collect the required name, mobile and email details; for a returning customer, show the matched profile details that may be confirmed/updated.
4. Customer verifies email OTP using the rules in Section 7.
5. System creates a Booking_ID for this visit and records Visit_Type = Online.
6. Customer enters party size and chooses whether to reserve a table. If reserving, show available reservation choices based on current table capacity and protected upcoming bookings.
7. Customer selects date/time and submits. Backend validates the slot again; frontend availability alone is not enough.
8. System creates a Table_Reservations record for the selected table(s) and sets the correct reservation/grace deadlines.
9. Customer browses menu, selects food, changes quantities and reviews the cart.
10. Customer submits the order. The system stores the order against the Booking_ID. Food preparation is not sent to Kitchen until the online customer has arrived and Staff has verified arrival and authorized the order.
11. At the restaurant, customer presses I'm Arrived within the 10-minute grace deadline.
12. Staff sees the arrival and verifies the customer. Staff verification must happen within 15 minutes after the digital arrival confirmation.
13. Staff calls the customer when a suitable table is ready. A called table is protected from being allocated to another party.
14. Staff seats the customer and updates the physical table to Occupied only when the customer is actually seated.
15. Staff authorizes the order for kitchen preparation. Kitchen receives the authorized order and updates preparation status.
16. Customer can track the order from the Customer Website.
17. Staff handles extra items or item changes when requested and authorizes any new preparation request.
18. When dining is complete, Staff prepares one final bill for the Booking_ID.
19. Customer pays using the restaurant's manual UPI process and provides the transaction reference/UTR as required.
20. Staff verifies that the money was actually received in the restaurant account before marking the payment as paid.
21. If needed, a refund request follows the refund process in Section 13.
22. Customer departs. Staff releases the table only after the party has left and the cleaning/turnover rule is followed.
23. Customer may submit one 1–5 star rating and optional comment for the completed Booking_ID.

9. Walk-in QR and queue journey

A walk-in customer scans the restaurant QR code. The QR opens the Customer Website with a walk-in source indicator. Walk-in customers are not forced to wait for a future reservation arrival process.

1. Customer scans the restaurant QR code. The URL opens the Customer Website with source=walkin.

2 Customer identifies/registers and verifies their email OTP as required.

3 System creates a new Booking_ID and records Visit_Type = Walk-in.

4 Customer enters party size and joins the queue if a suitable table is not immediately available.

5 Queue_Joined_At is recorded. Queue position is calculated dynamically; do not store a permanent queue position number.

6 Queue order is FIFO by Queue_Joined_At, with the capacity-fit rule described in Section 10.

7 Staff sees the walk-in queue and available tables. If a suitable table is ready, Staff calls the customer.

8 A called table is protected so another staff member or process cannot allocate it to someone else.

9 Staff seats the customer. Only then does the physical table status become Occupied.

10 Customer can place an order. For walk-in, staff authorization is permitted after seating.

11 Kitchen receives only staff-authorized preparation requests.

12 The same order, bill, payment, refund, departure and feedback rules apply.

Walk-in QR does not mean automatic table allocation. It identifies the visit source and starts the walk-in flow. Staff remains responsible for allocation and seating.

10. Table allocation and reservation rules

10.1 Table status

Table_Status	Meaning
Available	The table is physically available for allocation, subject to reservations and protection windows.
Occupied	Customers are physically seated at the table.
Unavailable	The table cannot be allocated, for example due to maintenance.

Do not add Reserved as a Tables.Table_Status value. Reservation is tracked separately in Table_Reservations.

10.2 Reservation grace and no-show

- The grace period is 10 minutes.
- For an immediate reservation, the grace deadline starts from the table allocation time.
- For a future reservation, the grace deadline starts from the scheduled reservation time.
- Customer must press I'm Arrived within the grace deadline.
- If the customer does not confirm arrival in time, the reservation expires/no-show is recorded and the reservation allocation is released.
- A reservation expiry must not change a physically occupied table to Available. Physical table state and reservation state are separate.

10.3 Staff verification and call protection

- After customer taps I'm Arrived, Staff must verify the arrival within 15 minutes.
- Staff call is a separate state from seating. A call does not mean the customer is seated.
- When a table is called/held for a customer, protect it from double allocation until seating, cancellation or the defined call-expiry handling.
- Set Tables.Table_Status = Occupied only when the customer is actually seated.

10.4 Blocking time and upcoming reservation protection

Use a 90-minute dining period plus 15 minutes for cleaning/turnover: a total 105-minute blocking window. Table availability calculations must protect upcoming reservations within this window. Do not offer a table for a slot that overlaps a protected booking or the required turnover period.

10.5 Capacity-fit queue rule

Queue is FIFO by Queue_Joined_At, but capacity-fit selection allows a smaller party to use a smaller suitable table while a larger party retains priority for a table that can fit them. The system must not blindly allocate the first free table if doing so blocks a larger waiting party from the only suitable table. The selection logic must consider party size, table capacity and upcoming reservation protection.

10.6 Multiple tables

One booking may reserve multiple tables. Create one Table_Reservations row per table linked to the same Booking_ID. The backend must validate that the full allocation is valid and avoid partial allocations if the complete request cannot be satisfied.

11. Food ordering and kitchen workflow

11.1 Menu and cart

- Menu items come from the Menu tab. Show only items that are active and available for ordering.
- Display Food_Name, Category, Description, Price and Image_URL where present.
- Customer can search/filter, add items, change quantity and remove items before submitting.
- The server must fetch the authoritative price from Menu. Never trust Unit_Price or Subtotal sent by the browser.
- Save the accepted item price as Unit_Price in Order_Items so later menu price changes do not rewrite old orders.

11.2 Order creation and authorization

- Every order is linked to a Booking_ID.
- A booking may have multiple orders, including additional orders during the same visit.
- Order creation and kitchen authorization are separate events.
- Online order preparation can be authorized only after the customer arrival has been verified by Staff.
- Walk-in order preparation can be authorized after the customer is seated.
- Staff authorization records Authorized_At and Authorized_By.
- Kitchen sees only authorized preparation requests.

11.3 Kitchen status

Kitchen staff opens the Kitchen Website, views the authorized queue, opens an order and updates its preparation status using only the allowed status transitions. Every meaningful change is recorded in Order_Status_History with time and actor. The exact full status vocabulary should be centralized in code and validated; do not let users type arbitrary status strings.

Kitchen does not change table status, customer arrival, reservation, bill totals, payment verification or refund approval.

12. Extra items, item changes and cancellation

During the visit, a customer may request more food or ask to change/cancel an item. Staff controls these changes so the kitchen receives accurate, authorized work.

- Extra food items create a new order or order items linked to the same Booking_ID, using the agreed order structure.
- Staff checks current menu availability and authoritative price before accepting the request.
- Staff authorizes new preparation requests before they appear in Kitchen.
- Order_Items contains Item_Status, Cancellation_Reason, Replacement_Food_ID and Price_Difference for item-level lifecycle and adjustment information.
- A cancellation or replacement must be recorded, not silently erased.
- If a replacement has a price difference, the backend calculates and records it using server-side menu prices.
- Already prepared or in-progress food must not be treated as if it can always be cancelled without cost. The exact policy for charging such cases requires owner confirmation.

13. Billing, payment, refund and departure

13.1 Final bill

- There is one final Bill_ID per Booking_ID.
- The bill includes all accepted order items for that visit, including extra items and valid adjustments.
- The backend calculates Subtotal from saved item quantities and unit prices.
- Tax_Amount, Discount and Grand_Total are calculated server-side.
- Discount_Reason and Adjustments_JSON preserve the reason/details of bill changes.
- Prevent duplicate final bills using a lock and a server-side check for an existing final bill.

13.2 Manual UPI payment

1. Customer sees the final bill and restaurant UPI payment instructions/QR.
2. Customer completes payment in their UPI app.
3. Customer submits the transaction reference/UTR and amount through the Customer Website or staff-assisted flow.
4. The system records the payment as pending verification; the customer's submitted reference is not proof of receipt.
5. Staff checks the restaurant's actual bank/UPI receipt outside the system.
6. Only after confirming receipt does Staff mark the payment as successful/paid.
7. If the reference is invalid, duplicate or amount does not match, Staff leaves it unverified and follows the rejection/correction process.

Do not integrate Razorpay, Stripe or another payment gateway in the initial scope. Do not automatically mark a payment paid just because a customer enters a UTR.

13.3 Refund workflow

1. A refund request may be full or partial.
2. Staff creates/forwards the refund request with amount and reason.
3. Admin reviews the request and approves or rejects it.
4. If rejected, store Rejection_Reason and reviewer/time.
5. If approved, the restaurant performs the payout manually outside the system.
6. Staff/Admin records completion details, completed amount, completion actor/time and refund reference.
7. Keep the original payment record immutable. Link refund records using Related_Payment_ID and the refund-related fields.

13.4 Departure and table release

- Staff confirms the customer party has departed.
- Staff releases the physical table only after the party leaves.
- Apply the 15-minute cleaning/turnover allowance as part of the 105-minute table blocking rule.
- After cleaning, set the physical table status to Available unless it is marked Unavailable for another reason.
- Do not release a table merely because a reservation expired if customers are physically seated.

14. Feedback, history and reports

14.1 Feedback

- Feedback is available for a completed Booking_ID.
- Rating is 1 to 5 stars.
- Comment is optional.
- Allow only one feedback submission per completed Booking_ID.
- Store Feedback_Rating, Feedback_Comment and Feedback_Submitted_At in Bookings. Do not create a separate Feedback tab.

14.2 History and audit

Bookings.Status_History_JSON can preserve the booking timeline. Order_Status_History stores order status changes with History_ID, Order_ID, Order_Status, Changed_At, Changed_By and Change_Reason. Important payment/refund and administrative changes should also have a safe audit trail using the approved schema and application logs; do not silently rewrite history.

14.3 Admin reports

- Daily/weekly/monthly booking and visit counts.
- Online versus Walk-in visit counts.
- Queue waiting and table utilization information.
- Orders and item sales by menu item/category.
- Revenue and bill totals, separated from unverified/pending payments.
- Payment verification and refund status summaries.
- Menu availability and operational exceptions.

Reports must calculate from saved records and clearly distinguish gross bill totals, verified payments, pending payments and completed refunds. Do not present pending UPI references as collected revenue.

15. UI and design requirements

15.1 Visual direction

Use the SAVOR app screenshot previously supplied as color inspiration, not as a screen-by-screen copy. The overall design should feel warm, clean, readable and suitable for a restaurant.

Purpose	Color
Warm page background	#FFF8F3
Primary burgundy / maroon	#650B20
Secondary mustard / gold	#D9AA49

Purpose	Color
Main text / charcoal	#292323
Soft beige / supporting surfaces	#E8D8C8

- Do not use green as the main brand color.
- Use readable typography, clear spacing and obvious primary actions.
- Use responsive layouts for desktop, tablet and mobile.
- Show loading, empty, success and error states; do not leave blank screens.
- Use clear status labels and accessible contrast.
- The customer-facing UI and all five websites are English-only unless the owner explicitly changes this requirement.
- For the restaurant project, explanatory conversation can be Tanglish/English in Latin script; do not place Tamil Unicode in the UI.

15.2 Role-specific navigation

Each website should show only the navigation relevant to that role. Main Website is public. Customer Website focuses on visit/order/bill history. Staff Website focuses on operations. Kitchen Website focuses on preparation. Admin Website focuses on management and reports. Backend permission checks remain mandatory.

16. Backend request lifecycle and data consistency

16.1 Standard request lifecycle

1. Frontend collects the user's action and sends a structured request to the Apps Script backend.
2. Code.gs accepts only a known action and validates the request structure.
3. For protected actions, AuthService validates the session.
4. Authorization checks role and record ownership.
5. Validation checks all fields and allowed state transitions.
6. The relevant service applies business rules.
7. Database.js opens the one configured spreadsheet and accesses the required tab.
8. Critical operations acquire a lock, re-read current state, validate again and write safely.
9. The service records required history/audit information.
10. Code.gs returns a consistent response with only the data the caller is permitted to see.
11. Frontend updates the screen and handles success, loading or error states.

16.2 Consistent response shape

Use one consistent response format, for example:

```
{ success: true, data: { ... }, message: "..." }
{ success: false, error: { code: "...", message: "Safe user-facing message" } }
```

This is a conceptual shape. Choose and document the final exact response contract once, then use it across all five websites. Do not return raw sheet rows, secrets or internal exception details.

16.3 Concurrency

- Two customers may request the same table at nearly the same time. Re-check availability inside a lock before writing a reservation.
- Two staff members may try to allocate the same table. Only one allocation may succeed.

- A queue call must atomically protect the selected table.
- Bill creation must be idempotent or guarded against duplicate final bills.
- Refund review/completion must reject invalid repeated transitions.
- Do not rely on a frontend button being disabled to prevent duplicate submissions.

17. Error handling and edge cases

Situation	Required behaviour
OTP expired	Reject verification and ask customer to request a new OTP.
OTP attempts exhausted	Block further attempts for that OTP; apply resend/new OTP rules.
OTP resend too soon	Reject until the 60-second cooldown passes.
Customer misses 10-minute grace	Expire/no-show the reservation and release its reservation allocation.
Customer confirms arrival but Staff does not verify in 15 minutes	Keep the state visible and apply an explicit timeout/escalation policy; do not silently mark seated.
No suitable table	Keep walk-in in queue; do not assign an undersized table.
Upcoming booking overlaps	Protect the table for the 105-minute blocking window.
Two requests allocate same table	Lock, re-check and allow only one successful allocation.
Menu item becomes unavailable	Reject new order/addition and explain the item is unavailable.
Customer changes browser-sent price	Ignore browser price; recalculate from Menu on server.
Kitchen receives unauthorized order	Do not show it in kitchen queue.
Duplicate final bill request	Return the existing final bill or safe duplicate error; do not create a second final bill.
UTR submitted but receipt not found	Keep payment unverified/pending; do not mark paid.
Refund rejected	Store reviewer, time and rejection reason; keep original payment unchanged.
Customer tries to view another customer's booking	Deny access after server-side ownership check.
Spreadsheet unavailable	Return a safe service-unavailable response; do not report a false successful write.

18. Testing and demo data

18.1 Demo accounts

Create exactly six demo user/customer profiles for testing: 1 Admin, 1 Staff, 1 Kitchen account and 3 Customer accounts. Do not create demo bookings, orders, bills or payments unless the owner approves them. Use safe test credentials and force appropriate password setup; do not hardcode production credentials.

18.2 Required end-to-end test scenarios

1. Online customer registration, email OTP, reservation, order, I'm Arrived, Staff verification, table call, seating, kitchen preparation, bill, manual payment verification, departure and feedback.
2. Walk-in QR registration, walk-in Booking_ID, queue, table allocation, seating, order, kitchen, bill, payment and departure.
3. Customer books without reserving a table and follows the allowed arrival/queue process.
4. Customer adds extra food items during the visit and Staff authorizes them.
5. No-show: customer does not confirm within the 10-minute grace period; reservation allocation is released.

6 Future reservation grace starts from scheduled reservation time.

7 Immediate reservation grace starts from allocation time.

8 Staff verification timeout after digital arrival is handled safely.

9 Two customers/staff attempt the same table; double booking is prevented.

10 Capacity-fit queue rule preserves larger-party suitability while allowing smaller parties to use a suitable smaller table.

11 Upcoming reservations are protected by the 105-minute block.

12 Kitchen cannot see or update an unauthorized order.

13 Customer cannot access another customer's records.

14 Invalid/duplicate UTR cannot be marked paid without staff receipt verification.

15 Partial and full refund requests go through Admin approval/rejection and manual completion.

16 Only one final bill exists for each Booking_ID.

17 Feedback cannot be submitted twice for the same completed Booking_ID.

18 Database schema verification detects missing/extra tabs or incorrect headers without deleting data.

18.3 Testing layers

- Unit tests for validation, OTP rules, price calculations, queue selection and state transitions.
- Service tests using a mock database adapter.
- Authorization tests for every role and customer ownership.
- Integration tests against the approved fresh Google Spreadsheet after authorization.
- Browser tests for all five interfaces at mobile and desktop sizes.
- End-to-end tests for the complete customer visit lifecycle.
- Concurrency tests for reservation, table allocation, bill finalization and refunds.

19. Build order and acceptance checklist

Build in controlled stages. Do not jump directly to all screens before the database contract and backend rules are stable.

Stage 1 — Project foundation

Create repository/local folders as appropriate, five website shells, shared design tokens, Apps Script module layout and documented request/response contract. No GitHub setup is required in current scope.

Stage 2 — Schema and mock database

Implement exact 12-tab/108-column schema definitions, mock adapter, schema verifier and safe test fixtures.

Stage 3 — Database connection module

Implement Config.js contract, Database.js getDatabaseConnection(), isDatabaseConfigured(), connection test and read-only verifyDatabase(). Keep live spreadsheet authorization for approved integration.

Stage 4 — Authentication and permissions

Implement customer OTP, staff/admin/kitchen login, password hashing, sessions, role checks and customer ownership.

Stage 5 — Customer and booking

Implement customer profile, Booking_ID generation, Online/Walk-in Visit_Type and booking lifecycle.

Stage 6 — Reservations, queue and tables

Implement availability, 10-minute grace, 15-minute staff verification, 105-minute blocking, capacity-fit FIFO queue, call protection and seating/release.

Stage 7 — Menu, orders and kitchen

Implement menu, cart/order creation, staff authorization, kitchen queue and status history.

Stage 8 — Item changes and billing

Implement extra items, cancellation/replacement handling, final bill, server-side calculations and duplicate prevention.

Stage 9 — Payments and refunds

Implement manual UPI reference capture, staff verification, full/partial refund request, admin review and manual completion.

Stage 10 — Feedback, reports and UI completion

Implement one-time feedback, history, admin reports, responsive design, empty/loading/error states.

Stage 11 — Integration and deployment

Owner creates/approves fresh spreadsheet and project, enters approved configuration and secret, authorizes Apps Script, runs read-only verification, tests actual reads/writes and deploys controlled web app access.

Stage 12 — Final validation

Run all acceptance tests, verify roles and audit trail, review security, test all five websites and document deployment/operations.

Definition of done

- All five websites work and share one backend and one spreadsheet.
- All 12 tabs and 108 columns match the schema exactly.
- Code.gs routes requests safely and business logic is modular.
- Google Sheets connection is proven by an actual authorized connection test, not just a configured ID.
- All authentication, role and ownership checks are enforced on the backend.
- Online and walk-in journeys work end to end.
- Queue, reservation, grace, table blocking and double-allocation rules pass tests.
- Kitchen sees only authorized work.
- One final bill per booking; payment is not marked paid until staff verifies receipt.
- Refund approval and completion are auditable.
- All demo and end-to-end scenarios pass; no unresolved critical security or data-integrity issue remains.

20. Decisions that must not be guessed

These points were not fully fixed in the retained project requirements. The AI agent must keep them configurable or ask the owner before final implementation. Do not silently invent a business policy.

Open decision	What needs owner confirmation
Tax rules	Tax rate, whether prices include tax, tax rounding and any tax breakdown.
Discount rules	Who may apply discounts, permitted limits, approval needs and stacking rules.

Open decision	What needs owner confirmation
Reservation deposit	Whether any advance payment/deposit is required. Current baseline does not require a gateway.
Exact status vocabulary	Finalize allowed booking, reservation, queue, order, item, payment and refund statuses and every permitted transition.
Staff verification timeout	What happens after the 15-minute verification window: alert, retry, cancellation or escalation.
Called-table expiry	How long a table remains protected after Staff calls a customer, and how cancellation/no-response is handled.
Cancellation charging	How to charge for food already prepared or in progress when an item is cancelled.
Session duration	Exact expiry duration and idle-session policy for customer and employee sessions.
Email delivery details	Approved sender identity, email wording and any deliverability settings.
Opening hours and reservation horizon	Operating hours, days closed, earliest/latest booking and advance-booking limit.
Refund time policy	Operational deadline for manual payout and customer communication.

The 10-minute arrival grace, 15-minute Staff verification window, 90-minute dining + 15-minute cleaning block, FIFO capacity-fit queue rule, manual UPI verification and Admin-controlled refund decision are already defined requirements. Do not reopen these as undecided items.

21. Final instructions to the AI coding agent

Build the exact system described in this document. Before coding, convert each section into a traceable checklist. Implement the backend contract and schema first, then authentication/authorization, then booking and table logic, then orders/kitchen, then billing/payments/refunds, then the five interfaces.

- Do not simplify away any of the five websites or role boundaries.
- Do not replace Google Apps Script or Google Sheets with another backend/database.
- Do not merge all backend code into Code.gs. Keep Code.gs as the controlled entry point and use modular services.
- Do not create a second spreadsheet for another website.
- Do not change the 12-tab/108-column schema without owner approval.
- Do not use browser-provided price, role, ownership or status as trusted values.
- Do not automatically mark UPI payment as paid based on a customer-entered UTR.
- Do not let Kitchen authorize its own work or manage billing/table allocation.
- Do not delete or overwrite existing owner files. Use only the fresh, explicitly approved restaurant project resources.
- Do not claim a live integration or deployment until it has actually been tested.

Expected final output: a working dine-in restaurant management system with five separate responsive websites, one modular Google Apps Script backend, one shared Google Spreadsheet database, secure role-based access, complete online and walk-in customer journeys, queue/table optimization, authorized kitchen workflow, final billing, manual payment verification, controlled refunds, feedback, history and admin reports.

End of complete project blueprint.